SafeSight – Demo Script and Viva Guide

YOLO11s construction-site PPE and safety-event monitoring · prepared 17 October 2026 from your notebook (DL_17_10.ipynb), training figures and SafeSight_Project_Report.docx

0. How to use this guide

Part A is what you say in the demo (a slide-by-slide speaking script). Part B is the knowledge base: motivation, research gap, literature, dataset, model, training, evaluation, results. Part C is the file-by-file explanation. Part D is viva questions with answers and the cross-questions a faculty member is likely to follow up with.

Two honesty rules run through everything below. First, anything I could read directly in your notebook log or figures is stated as fact. Second, anything about the Flask app, tracker, zones and dashboard comes from your project report only, because the application source files were not uploaded; those places are marked **(confirm in your code)**. Never claim in the viva something you did not personally build or run.

1. Numbers you must know cold

Item	Value
Dataset	Roboflow project safety-object-detection-app / construction-site-safety-ccp7o, version 1, licence CC BY 4.0, exported in YOLO format
Images	1,819 train / 518 validation / 260 test = 2,597
Classes (10)	Hardhat, Machinery, Mask, NO-Hardhat, NO-Mask, NO-Safety Vest, Person, Safety Cone, Safety Vest, Vehicle
Labelled objects	about 25.6 thousand in train (sum of the bars in labels.jpg), 7,997 in validation, 3,733 in test
Model	YOLO11s, fine-tuned from pretrained yolo11s.pt
Size	181 layers, 9.43 M parameters, 21.7 GFLOPs while training; fused for inference: 100 layers, 9.42 M parameters, 21.4 GFLOPs; best.pt is 19.2 MB

Item	Value
Training	50 epochs, image size 640, batch 16, SGD, lr0 0.01, lrf 0.01, momentum 0.937, weight decay 0.001, seed 0, workers 8, AMP on
Where	Google Colab, Tesla T4 GPU (14.9 GB), Google Drive for outputs
Training time	0.445 hours, about 27 minutes
Validation result (518 images)	Precision 0.891, Recall 0.728, mAP@0.5 0.798, mAP@0.5:0.95 0.561
Test result (260 images)	Precision 0.889, Recall 0.737, mAP@0.5 0.805, mAP@0.5:0.95 0.567
Best operating point	F1 0.80 at confidence 0.361
Recall ceiling	0.85 (recall at confidence 0)
Best / worst class (validation AP@0.5)	Machinery 0.926 / Safety Cone 0.596
Speed on the T4 (batched evaluation)	inference 5.9 ms per image on validation and 11.4 ms on test, plus 2–3 ms pre/post-processing
Software in the log	Ultralytics 8.4.179, PyTorch 2.11.0 with CUDA 13.0 build, Python 3.13.15, roboflow, PyYAML

PART A – DEMO SCRIPT (about 10 minutes)

Speak in your own words; the text below is a script to rehearse, not to read aloud. Adjust button and page names to match your actual interface.

Slide 1 – Title and problem (0:45)

“Good morning. My project is SafeSight, a camera-based safety monitor for construction sites. A construction site has moving workers, machines and vehicles, and a supervisor cannot watch every corner for the whole shift. Missing hardhats or vests and people standing in the wrong zone are easy to overlook. SafeSight uses a deep-learning detector to look at camera footage and flag these conditions, and it keeps evidence so a human can review each alert.”

Slide 2 – Motivation (0:45)

“Manual supervision is labour-intensive and error-prone, especially over a large site. Earlier research already showed that cameras plus deep learning can detect PPE. But a detector only answers a per-frame question: what objects are in this image. A site manager needs more: who is this person over time, are they inside a restricted zone, for how long, is the required PPE present, and what is the proof. That gap between detection and a usable monitoring workflow is what I worked on.”

Slide 3 – Literature and research gap (1:00)

“I surveyed eight papers. Delhi et al. showed YOLOv3 can classify PPE compliance with alarms and reports. Wang et al. compared eight YOLO detectors on real construction images. Zaidi et al. used temporal analysis to cut false alarms. My base paper is Cheng et al. 2022, which combines worker re-identification with PPE classification and logs incidents of workers without PPE. They report improving re-identification accuracy by 4 percent and PPE classification accuracy by 13 percent on a real site dataset. I do not reproduce their cross-camera re-identification; I use single-camera tracking. The gap I address is narrow: a single-camera prototype that combines user-drawn zones, a dwell-time rule, per-person PPE evidence, evidence snapshots, an after-hours rule and a live dashboard, and a clear plan to measure it at event level, not just by detector accuracy.”

Slide 4 – Dataset (0:45)

“I used a public Roboflow dataset, version 1, licensed CC BY 4.0, in YOLO format. It has 2,597 images: 1,819 for training, 518 for validation and 260 for testing. There are ten classes, including both the positive and the negative PPE states: Hardhat and NO-Hardhat, Mask and NO-Mask, Safety Vest and NO-Safety Vest, plus Person, Machinery, Vehicle and Safety Cone. The classes are imbalanced: Person has about 6,700 labelled boxes while Vehicle has about 1,000. Most boxes are small, under 10 percent of the image width, which makes this a small-object problem.”

Slide 5 – Model and training (1:15)

“I chose YOLO11s, the small variant of Ultralytics YOLO11. It is a single-stage, anchor-free detector, small enough, about 9.4 million parameters, to run in real time on modest hardware, and I started from COCO-pretrained weights so I only needed to fine-tune. I trained in Google Colab on a Tesla T4 for 50 epochs at 640 pixels, batch 16, with SGD,

learning rate 0.01 decaying linearly to 0.0001, momentum 0.937 and weight decay 0.001. Training took about 27 minutes. Standard Ultralytics augmentation was on, including mosaic, which is switched off for the last ten epochs."

Slide 6 – Results (1:30)

"On the validation set the model reached precision 0.891, recall 0.728, mAP at 0.5 of 0.798 and mAP at 0.5 to 0.95 of 0.561. On the held-out test set it reached 0.889, 0.737, 0.805 and 0.567, so there is no sign of overfitting. The best confidence threshold from the F1 curve is about 0.36. Machinery is the strongest class at 0.926 AP, and Safety Cone and NO-Mask are the weakest at 0.596 and 0.691. The curves in results.png are still improving at epoch 50, so the model is under-trained rather than over-fitted. I want to be clear that these are detector metrics. They do not prove the event-level accuracy of zone alerts, which I list as future evaluation."

Slide 7 – System pipeline (1:15)

"Here is the full pipeline. A frame comes from a webcam, video or uploaded image. YOLO11s detects objects. ByteTrack gives each person a temporary ID within that camera. I test each person's position against the polygon zones the user drew and accumulate dwell time; after a 1.5-second threshold an event is created. PPE detections are matched to the person by spatial overlap and turned into present, missing or uncertain. An after-hours rule raises an event if someone is detected after the configured closing time. Each event stores time, zone, track ID, PPE status and a snapshot, and the Flask dashboard shows it live." **(confirm numbers and names in your code)**

Slide 8 – Live demo (2:00)

Follow this order and keep the same order every rehearsal.

1. Open the dashboard and show the live or uploaded video with boxes and labels.
2. Point out two or three classes appearing (Person, Hardhat or NO-Hardhat, Safety Vest).
3. Show or draw a zone, walk or play a clip of a person entering it, and let the dwell timer pass.
4. Show the event row that appears: timestamp, zone, track ID, PPE status and the evidence snapshot.
5. Show the after-hours setting and what it does.

6. Say one sentence about a limitation you observed (for example a missed small item or an ID change after occlusion). Faculty trust a team that points out its own limits.

Fallback: keep a recorded screen capture of a successful run, and one saved image with predictions, in case the camera, network or Colab session fails.

Slide 9 – Limitations and future work (0:45)

"Limitations: ByteTrack IDs are only valid inside one camera and can change after occlusion; PPE status is inferred from boxes and can be uncertain when items are small or hidden; weak classes like Safety Cone need cautious thresholds; and I have not yet measured event-level precision, recall, false alarms per hour or alert latency. Future work is to annotate video, measure those quantities, train longer and with a higher image size, and add temporal confirmation to reduce false alarms."

Slide 10 – Conclusion (0:20)

"SafeSight is an integration prototype: a YOLO11s detector with 0.80 mAP at 0.5 on ten classes, connected to tracking, zones, dwell rules, PPE evidence and a dashboard. Its main contribution is the workflow and the honest plan to evaluate it. Thank you; I am happy to take questions."

PART B – KNOWLEDGE BASE

2. How the project started: motivation, problem statement and research gap

The story in four steps (use this when asked "how did you start").

1. **Problem.** Construction is a high-risk industry. PPE such as hardhats and high-visibility vests reduces injuries, but checking compliance by hand across a large site over a long shift is inconsistent.
2. **Literature survey.** I read papers on PPE detection and site-safety monitoring and found that detection accuracy is well studied, while connecting detections to a person over time, to zones and to logged evidence is studied less as a complete workflow.
3. **Technical choice.** I fine-tuned a pretrained YOLO11s detector on a public ten-class construction-safety dataset in Colab, because that is fast, reproducible and free of the cost of collecting and labelling thousands of images.

4. **System.** I wrapped the detector in a monitoring layer (tracking, zones, dwell rule, PPE assessment, evidence snapshots, dashboard, after-hours rule) so the output is an alert a supervisor can act on, not only boxes on an image.

Problem statement. Frame-level PPE detectors do not by themselves tell a supervisor who is in which zone, for how long, with what PPE, and with what evidence; this project builds and evaluates such a pipeline for a single camera.

Research gap (say it narrowly and precisely). Existing work already covers PPE detection [1,2,4,5,7,8], person-linked PPE monitoring with incident logging [3] and temporal false-alarm reduction [6]. Therefore the gap is not "nobody has done alerts." The gap is: a single-camera, locally deployable prototype that combines user-configured polygon zones, a dwell-time rule, per-track PPE evidence, evidence snapshots, a configurable closing-time rule and a live dashboard, **together with event-level evaluation** of that exact combination (zone-entry recall, PPE-to-person assignment accuracy, track continuity, false alerts per camera-hour, after-hours recall and alert latency). Your report states that this event-level evaluation is not yet done; say so yourself before a faculty member does.

What you must not claim. Do not claim a new architecture, state-of-the-art accuracy, worker identification, or a validated safety product. Do not compare your mAP of 0.80 with other papers' numbers as if it were the same test; datasets and tasks differ.

3. Literature review and the base paper

3.1 Base paper in detail – Cheng et al. (2022) [3]

Citation. J. C. P. Cheng, P. K.-Y. Wong, H. Luo, M. Wang and P. H. Leung, "Vision-Based Monitoring of Site Safety Compliance Based on Worker Re-Identification and Personal Protective Equipment Classification," *Automation in Construction*, vol. 139, article 104312, July 2022 (published online 6 May 2022). DOI 10.1016/j.autcon.2022.104312.

Problem they address. On-site accidents are costly, and unsafe behaviour such as missing PPE is hard to catch because safety officers cannot continuously watch workers scattered over a large site. Workers also move between camera views, so continuous tracking across a wide area is needed for comprehensive monitoring.

Their idea. A framework that combines two deep-learning tasks. **Worker re-identification (ReID)** recognises that a worker seen in one camera view or time is the same person seen in another. **PPE classification** decides whether that worker is wearing the necessary PPE.

Combining both, a workflow logs each incident of a worker not wearing the required PPE, so the record is tied to a particular worker over time.

Their reported results. On an actual construction-site dataset their methods improved worker ReID accuracy by about 4 percent and PPE classification accuracy by about 13 percent compared with the baselines they tested.

Verified from public abstract only. I could confirm the points above from the paper's abstract and publisher record. Before the viva, read the paper's methodology and experiment sections yourself and note three things for your slides: which networks they used for detection, ReID and PPE classification; how large their dataset was; and what baselines the 4 percent and 13 percent improvements are measured against. A faculty member who has read the paper will ask exactly these.

Why you chose it as the base paper. It is the closest system-level work: it links a person's observed PPE state to a logged incident, which is the same logic as your event record.

What you adopt, what you change, what you do not do.

- *Adopted:* the idea of linking each person's observed PPE state to an incident record, rather than only drawing boxes.
- *Changed:* you use a one-stage detector with explicit NO-Hardhat, NO-Mask and NO-Safety Vest classes plus rule-based spatial association, instead of their separate PPE classification network.
- *Not done:* cross-camera re-identification. ByteTrack IDs are local to one camera stream and are temporary, not worker identities.
- *Added:* user-drawn polygon zones, dwell time, an after-hours rule, evidence snapshots and a live dashboard.

3.2 Supporting papers

[1] Delhi, Sankarlal and Thomas (2020), Frontiers in Built Environment 6:136. Applied transfer learning to YOLOv3 to classify workers by hardhat and safety jacket into four categories: SAFE, NOT SAFE, NoHardHat and NoJacket. Reported F1 of 0.96, with average precision and recall about 96 percent on their test set, and quasi-real-time operation on video from construction sites. *Use in viva:* the operational precedent for PPE alerts and reports. *Limit:* only two PPE items, no tracking or zones.

[2] Wang et al. (2021), Sensors 21(10):3478. Trained and compared eight YOLO-based detectors for six classes (helmets in four colours, person and vest) on their own CHV dataset of 1,330 images. YOLOv5x had the best mAP (86.55 percent) and YOLOv5s the

highest speed (52 FPS on GPU). Detection of helmet classes dropped by about 7 percent on blurred faces. *Use in viva:* evidence of the accuracy–speed trade-off, which supports choosing a small "s" model, and of blur sensitivity.

[4] Karlsson et al. (2023), ICPRAM. Studied several PPE classes and entry control on industry workers; per your survey, performance changed with camera distance. *Use in viva:* distance and small-object limits, relevant to your Safety Cone and NO-Mask weaknesses. (Skim the paper for exact numbers.)

[5] Al-Khiami and ElHadad (2024), EC3. A custom Ultralytics YOLOv8 PPE model for construction compliance. *Use in viva:* precedent for training with the Ultralytics framework as you did.

[6] Zaidi et al. (2024), Buildings 14(6):1878. Argues that frame-by-frame classification triggers false alarms from misdetections. Their system has three modules (object detection, rule compliance, temporal analysis); the temporal module examines several frames in a time window and raises an alarm only when a hazard threshold is exceeded. They report about 95 percent accuracy and F1 in scene classification and an average 2.03 percent reduction in false alarms across five test videos. *Use in viva:* justification for your dwell threshold and for the idea that one-frame detections should not trigger alerts.

[7] Tong et al. (2025), PLOS ONE 20(5):e0322115. MFD-YOLO: multi-scale PPE detection with model pruning for efficiency. **[8] Li et al. (2025), Frontiers in Built Environment 11:1563483.** SD-YOLOv5: improved detection of small and occluded PPE on construction sites. *Use in viva:* both show that small-object accuracy and efficiency are active research directions and name architectural fixes you list as future work. (Skim the abstracts for their numbers.)

3.3 Concept references you may be asked about

These are standard sources from general knowledge; verify the citation details before putting them on a slide.

- Redmon et al., "You Only Look Once," CVPR 2016 – single-stage detection.
- Jocher and Qiu, Ultralytics YOLO11, 2024 – the model you used.
- Zhang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," ECCV 2022 – the tracker.
- Lin et al., "Microsoft COCO," ECCV 2014 – the dataset behind the pretrained weights.
- Li et al., "Generalized Focal Loss," NeurIPS 2020 – source of the distribution focal loss (DFL).

- Zheng et al., “Distance-IoU Loss,” AAAI 2020 – CIoU box loss.
- Feng et al., “TOOD: Task-aligned One-stage Object Detection,” ICCV 2021 – task-aligned label assignment.
- Bochkovskiy et al., YOLOv4, 2020 – mosaic augmentation.

4. Dataset

Source and format. The notebook downloads version 1 of the Roboflow project safety-object-detection-app / construction-site-safety-ccp7o with the `roboflow` Python package, asking for the “yolov11” export format (the standard Ultralytics YOLO format). The API key is read from Colab Secrets with `userdata.get("ROBOFLOW_API_KEY")`, so it is never typed into the notebook – a good practice to mention. The licence is CC BY 4.0, so you must credit the dataset authors on your slide.

What is in the download. One image plus one text label file per sample. Each line of a label file is: class index, x-centre, y-centre, width, height, all normalised to the range 0–1. A `data.yaml` file lists the train, val and test image folders, `nc: 10` and the ten class names. The notebook extracted 5,206 files, which is about two files per image plus configuration and metadata.

Split. 1,819 train, 518 validation and 260 test images, which is almost exactly 70 percent, 20 percent and 10 percent. Train is used to update weights; validation is looked at after every epoch to track progress and pick best.pt; test is held out and used once at the end.

Cleaning observations from the log. Ultralytics scanned the labels and reported 0 corrupt images and removed one duplicate label in each of four training images. It also reported “0 backgrounds,” meaning every image contains at least one labelled object. So the model never saw an image with nothing in it; that can encourage false positives on empty scenes (this is my inference, not something you measured).

Classes. Hardhat, Machinery, Mask, NO-Hardhat, NO-Mask, NO-Safety Vest, Person, Safety Cone, Safety Vest, Vehicle. Note the design: violations are **explicit classes**. The model is trained to find a head with no hardhat (NO-Hardhat) rather than inferring a violation from the absence of a Hardhat box.

- *Advantage:* a violation is detected directly, and a missing detection is not automatically treated as a violation.
- *Disadvantage:* the negative classes depend on subjective labelling (hair, cap, hood, face turned away) and are visually subtler, which is why NO-Mask has the second-lowest AP.

4.1 Reading labels.jpg (the four panels)

1. **Class counts (top left).** Person 6,715; Machinery 3,594; NO-Safety Vest 2,701; Hardhat 2,345; NO-Mask 2,268; Safety Vest 2,198; Safety Cone 1,902; NO-Hardhat 1,607; Mask 1,240; Vehicle 1,065. The imbalance between the largest and smallest class is about 6.3 to 1.
2. **Overlaid boxes (top right).** All training boxes drawn from a common centre, showing a wide range of box sizes with a dense core of small boxes.
3. **Centre-position heat map (bottom left).** Object centres are not spread evenly: there are four dense blobs near the quarter points of the image (roughly 0.25 and 0.75 on both axes). I cannot tell from the plot alone why. One possible explanation is that many source images are tiled or collaged 2-by-2 composites; another is how the dataset was assembled. Open ten random training images before the viva so you can answer this honestly.
4. **Width–height heat map (bottom right).** Almost all of the density is in the bottom-left corner, meaning most boxes are under about 10 percent of the image width and height. At 640 pixels that is under roughly 64 pixels, so this is largely a small-object problem.

Important insight for the viva. Class count does not predict accuracy. Safety Cone has 1,902 boxes (more than Mask with 1,240) but the lowest AP, 0.596, while Mask has an AP of 0.892. So difficulty here comes from object size, appearance and context, not just the number of examples.

5. The model: YOLO11s and why it was chosen

5.1 What object detection means

For every image the model must output a set of boxes, each with a class label and a confidence score. It is more than classification (one label per image) and different from segmentation (a label for every pixel).

5.2 What YOLO11 is

YOLO11 is the 2024 generation of Ultralytics' YOLO ("You Only Look Once") family. It is a **single-stage** detector: one network pass over the whole image predicts boxes and classes together, with no separate region-proposal step, which is why it is fast. The "s" suffix means the small size in a family of n, s, m, l, x.

Architecture in three parts.

- **Backbone** extracts features. Your log shows the stem and stages of strided convolutions, `C3k2` blocks (a faster CSP-style feature block; the log line reads `C3k2 [64, 128, 1, False, 0.25]`), a spatial pyramid pooling block and, in YOLO11, a partial self-attention block (C2PSA) that helps the model use wider context.
- **Neck** fuses features from three scales by upsampling and concatenation, so small objects use fine detail and large objects use deeper context.
- **Head** is **decoupled** (separate branches for box regression and for class scores) and **anchor-free** (it predicts box distances from each grid cell rather than adjusting pre-defined anchor boxes). At a 640 input it predicts on 80×80, 40×40 and 20×20 grids, i.e. 8,400 candidate predictions per image, which are then filtered by confidence and non-maximum suppression (NMS).

A detail you can quote from your log. The training summary shows 9,431,662 parameters but 9,431,646 with gradients – a difference of 16. The log says "Freezing layer model.23.dfl.conv.weight": that layer holds 16 fixed weights used to turn the predicted distribution over 16 bins into a box coordinate. It is intentionally not trained.

Losses. The log lists the loss gains box = 7.5, cls = 0.5 and dfl = 1.5.

- *Box loss:* a CIoU-type loss that penalises overlap error, centre distance and aspect-ratio mismatch.
- *DFL (distribution focal loss):* trains each box edge as a probability distribution over bins, giving finer localisation.
- *Class loss:* binary cross-entropy on class scores.
- *Label assignment:* a task-aligned assigner chooses which grid cells count as positives for each ground-truth object using a mix of class score and IoU.

5.3 Transfer learning (why you did not start from scratch)

You loaded `yolo11s.pt`, which was pre-trained by Ultralytics on the 80-class COCO dataset. The log says "Overriding model.yaml nc=80 with nc=10" and "Transferred 499/499 items from pretrained weights," so every layer's weights were loaded, and "Remapped 1/10 cls head rows from pretrained weights by class name," which I read as the Person class being initialised from COCO's person (the only name in common). Pre-trained backbones already know edges, textures, shapes and people, so with only 1,819 training images the model reaches mAP@0.5 of 0.41 after just one epoch and 0.80 after fifty. Training from random weights would need far more data and time.

5.4 Why YOLO11s and not something else

- **Speed and real-time use.** A site monitor must process live video. Single-stage YOLO models are built for this; Wang et al. [2] show the same accuracy–speed trade-off among YOLO sizes.
- **Small size.** About 9.4 M parameters and 21.4 GFLOPs fused, so it trains in about 27 minutes on a free-tier T4 and can run on modest hardware.
- **Larger models** (m, l, x) are usually more accurate but slower, need more GPU memory, and risk over-fitting on a 1.8 thousand image dataset. A smaller model (n) is faster but typically less accurate on small objects.
- **Two-stage detectors** such as Faster R-CNN are strong but heavier and slower for live video.
- **Newer than YOLOv5/YOLOv8** used in earlier PPE papers [2,5]; Ultralytics reports YOLO11 gives better accuracy for its size than YOLOv8.
- **One library for everything.** Ultralytics gives training, validation, plots, export and built-in tracking (ByteTrack) in a single API, which shortens integration.

Be honest here: you did not run your own model-comparison experiment. The choice is justified by the literature and by design constraints (real-time, small, pretrained, integrated tracker), not by an ablation of your own. If asked “what if YOLOv8s or YOLO11m were better,” say that is a sensible future experiment, and that you would compare mAP, small-class AP, FPS and memory on the same split.

6. How the model was trained – notebook walk-through (DL_17_10.ipynb)

Cell	What it does	Why it matters
0	`pip install ultralytics`; mounts Google Drive at /content/drive	Installs the framework; Drive keeps weights safe after the Colab session ends
1	Installs `roboflow` (and ultralytics again)	Needed to download the dataset; the second install of ultralytics is redundant
2	Reads the API key from Colab Secrets and creates a Roboflow client	Keeps the secret out of the code

Cell	What it does	Why it matters
3	Opens the workspace and project, selects version 1, downloads in YOLO format, prints data.yaml	Gets the data; confirms nc = 10 and the class names
4	Parses data.yaml with PyYAML and counts images per split	Verifies the 1,819 / 518 / 260 split
5	Re-checks that data.yaml exists	Sanity check (duplicates cell 4's purpose)
6	Loads `yolo11s.pt` and calls `model.train(...)` with the hyperparameters in section 7	The actual training: 50 epochs in 0.445 hours
7	Checks that the run folder and best.pt exist in Drive	Confirms the checkpoint was saved
8	Loads best.pt, runs `val` on `split="test"` and prints precision, recall, mAP@0.5 and mAP@0.5:0.95	Final, unbiased evaluation on unseen test images
9	Mounts Drive a second time at /content/gdrive	Prepares a backup location
10	Copies the whole run folder to a backup path and checks best.pt	Protects the trained model from being lost

What happens inside `model.train()`.

1. The YOLO11s architecture is built with 10 output classes and the pretrained weights are loaded.
2. Automatic mixed precision (AMP, 16-bit where safe) is checked and passes, which speeds training and saves GPU memory.
3. Training and validation data loaders are built. Each training image goes through augmentation before being fed to the network.
4. The optimiser is created: SGD with three parameter groups (81 weights with no decay, 88 weights with decay 0.001, 87 biases with no decay).
5. For each epoch: forward pass, loss computation (box, class, DFL), backward pass, and an optimiser step. The learning rate is warmed up for the first 3 epochs and then decays linearly.

6. After every epoch the model is evaluated on the 518 validation images and the metrics are logged to results.csv and plotted.
7. `last.pt` is saved every epoch and `best.pt` is saved whenever the fitness score improves (fitness = 0.1 × mAP@0.5 + 0.9 × mAP@0.5:0.95 by default).
8. In the last 10 epochs, mosaic augmentation is switched off (`close_mosaic=10`).
9. After epoch 50 the optimiser state is stripped from both checkpoints (19.2 MB each) and best.pt is validated once more.

Derived numbers (default Ultralytics behaviour; useful if asked about iterations).

- 1,819 images ÷ batch 16 gives 114 batches per epoch, so about 5,700 batches over 50 epochs.
- The nominal batch size `nbs` is 64, so gradients are accumulated over 4 batches before each weight update. That gives roughly 1,425 weight updates in total, which is small and is one reason the curves are still rising.
- Warm-up lasts 3 epochs (about 342 iterations).
- The weight-decay scale factor works out to 1 because batch × accumulation = 64 = nbs.
- Linear schedule: the learning rate goes from 0.01 to 0.01 × 0.01 = 0.0001 by epoch 50.

Where it was trained. Google Colab, GPU: Tesla T4 with 14,913 MiB (from the log), Python 3.13.15, PyTorch 2.11.0 with a CUDA 13.0 build, Ultralytics 8.4.179. Results were written to /content/drive/MyDrive/new/construction_yolo11s_v1 (best.pt, last.pt, results.csv, plots).

7. Hyperparameters – what each one means and why it was set that way

Parameter	Value	Meaning	Reason / effect
model	yolo11s.pt	Small YOLO11 with COCO weights	Real-time, pretrained, small data
epochs	50	Full passes through the training set	Enough to reach about 0.80 mAP@0.5 in about 27 min; curves had not plateaued, so more epochs would probably help

Parameter	Value	Meaning	Reason / effect
imgsz	640	Images resized so the longer side is 640 px	Standard YOLO size; a larger size (800–960) would help small objects but cost speed and memory
batch	16	Images per step	Fits a 15 GB T4 with AMP; small batches add gradient noise but are fine with accumulation
optimizer	SGD	Stochastic gradient descent with momentum	The classic YOLO recipe, generalises well; you chose it explicitly instead of letting Ultralytics pick automatically
lr0	0.01	Initial learning rate	Standard for SGD on YOLO
lrf	0.01	Final LR = lr0 × lrf = 0.0001	Gradual decay so late epochs make fine adjustments
momentum	0.937	Fraction of previous update kept	YOLO default; smooths and speeds up descent
weight_decay	0.001	L2 regularisation strength	Twice the usual default of 0.0005; a deliberate stronger regulariser for a small dataset
warmup_epochs	3 (default)	Epochs with ramped-up learning rate	Avoids early instability; the dip at epoch 4 is probably related

Parameter	Value	Meaning	Reason / effect
seed / deterministic	0 / True	Fixed random seed	Reproducibility; but this is one run, so no confidence interval
workers	8	Data-loading processes	Keeps the GPU fed
amp	True (default)	Mixed precision	Faster, less memory
patience	100 (default)	Early-stopping epochs	Larger than 50, so early stopping never triggered
pretrained	True	Use pretrained weights	Redundant because the model was already loaded from yolo11s.pt
plots	True	Save curves and matrices	Produces results.png, PR/F1/P/R curves and confusion matrices
mosaic	1.0 (default)	Probability of mixing 4 images into one	Gives varied scale and context; very effective for small objects
close_mosaic	10 (default)	Turn mosaic off for the last 10 epochs	Lets the model fine-tune on realistic, un-mixed images; visible as the drop in training losses near epoch 41
hsv_h / hsv_s / hsv_v	0.015 / 0.7 / 0.4	Colour jitter	Robust to lighting and colour of PPE
translate / scale	0.1 / 0.5	Shift and zoom	Robust to position and size
fliplr / flipud	0.5 / 0	Horizontal / vertical flip probability	Mirrored scenes make sense; upside-down ones do not
mixup / degrees / shear / perspective	0	Not used	Defaults

Parameter	Value	Meaning	Reason / effect
erasing / auto_augment	0.4 / randaugment	Random erasing and automatic policy	Robust to occlusion
albumentations	Blur, MedianBlur, ToGray, CLAHE at p = 0.01	Light extra image effects	Defaults
box / cls / dfl	7.5 / 0.5 / 1.5	Loss weights	Defaults; box accuracy is weighted most
val conf / iou	0.001 / 0.7	Thresholds used during validation	Very low confidence so the full precision–recall curve can be drawn

A trap to prepare for. The log shows `augment=False`. That flag is **test-time augmentation for inference**, not training augmentation. Training augmentation is controlled by mosaic, hsv, scale and the other parameters above, and it was on.

What you did not do. No hyperparameter search, no multiple seeds, no cross-validation, no class-weighting or oversampling, no model comparison. Say this plainly and offer it as future work.

8. Evaluation techniques

Intersection over Union (IoU). The area of overlap between a predicted box and a ground-truth box divided by the area of their union. A prediction counts as correct (true positive) if the class matches and the IoU is above a threshold (0.5 for mAP@0.5).

Terms. True positive (TP): correct detection. False positive (FP): a detection with no matching object or wrong class. False negative (FN): an object the model missed.

Precision = TP ÷ (TP + FP). Of the things the model reported, how many were right. High precision means few false alarms.

Recall = TP ÷ (TP + FN). Of the real objects, how many the model found. High recall means few misses.

F1 = 2 × P × R ÷ (P + R). The harmonic mean; the F1–confidence curve shows which confidence threshold balances the two.

Confidence threshold. The score a box must exceed to be kept. Raising it increases precision and lowers recall; this is exactly what the P, R and F1 curves show.

Average precision (AP). For one class, sort detections by confidence, trace the precision–recall curve and take the area under it (Ultralytics uses a 101-point interpolation). **mAP** is the mean of AP over all classes.

- **mAP@0.5:** AP with IoU threshold 0.5, i.e. a loose "found it roughly in the right place" test.
- **mAP@0.5:0.95:** the average of AP over IoU thresholds 0.50, 0.55, … 0.95 (ten values), which rewards precisely located boxes. It is always lower than mAP@0.5.

Non-maximum suppression (NMS). Removes duplicate boxes of the same object, keeping the highest-confidence one among boxes that overlap more than an IoU limit (0.7 in your log).

Confusion matrix. Rows are predicted classes and columns are true classes (in your figures). The extra "background" row counts objects the model missed and the "background" column counts detections with no real object (false alarms).

Validation versus test. Validation is evaluated after every epoch and is used to select best.pt, so it is slightly "seen" by the training process. Test is used once at the end and tells you how the final model generalises. Your test mAP@0.5 (0.805) is similar to or slightly higher than validation (0.798), which is evidence against over-fitting; but the test set has only 260 images, so per-class test numbers are noisy.

What these metrics do not measure. They measure single images. They do not measure tracking quality, correct zone membership, correct matching of a PPE box to a person, false alerts per hour, or alert delay. The proposed event-level metrics are: event precision, recall and F1; missed-event rate; false alerts per camera-hour; ID switches and track fragmentation; PPE-to-person association accuracy; after-hours detection recall; and alert latency. Threshold tuning must be done on validation data only so the test data stays untouched.

9. Results explained figure by figure

9.1 Overall numbers

Split	Images	Objects	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Validation (best.pt)	518	7,997	0.891	0.728	0.798	0.561

Split	Images	Objects	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Test (best.pt)	260	3,733	0.889	0.737	0.805	0.567

The training table shows 0.799 for validation mAP@0.5 at epoch 50 and the final re-validation of best.pt shows 0.798; they are the same number to rounding, so quote 0.80. Both splits have F1 of about 0.80 (computed from P and R). Precision and recall in Ultralytics tables are reported at the confidence level that maximises F1.

9.2 Per-class results

Class	Val AP@0.5	Test AP@0.5	Test recall	Test AP@0.5:0.95
Machinery	0.926	0.924	0.869	0.788
Mask	0.892	0.934	0.909	0.708
Person	0.888	0.901	0.829	0.681
NO-Safety Vest	0.851	0.859	0.783	0.597
NO-Hardhat	0.782	0.833	0.786	0.559
Vehicle	0.755	0.812	0.742	0.614
Hardhat	0.808	0.669	0.590	0.444
Safety Vest	0.795	0.766	0.677	0.523
NO-Mask	0.691	0.726	0.670	0.419
Safety Cone	0.596	0.626	0.521	0.336

What to say about it.

- Precision is uniformly high (0.83 to 0.93 in every class on both splits) while recall varies from about 0.51 to 0.91. So the model's weakness is **missing** objects, not inventing them. For a safety system that matters: a missed violation is worse than a false alarm, so the operating threshold should lean toward recall for important classes.
- Large, distinctive objects (Machinery, Person) and Mask are easiest. Small, thin or visually ambiguous ones (Safety Cone, NO-Mask, Hardhat versus NO-Hardhat) are hardest.

- **Hardhat is the surprise on the test set**: AP@0.5 falls from 0.808 (val) to 0.669 (test) and recall to 0.59. Possible reasons, which you should present as hypotheses and not as facts: a small test sample (310 hardhat instances in 260 images), different image conditions in the test split, and the Hardhat/NO-Hardhat confusion shown in the matrix. Do not hide this; faculty respect candour.
- mAP@0.5:0.95 is much lower than mAP@0.5 (0.561 versus 0.798): the model usually finds objects but does not always place the box tightly, which is expected for small objects at 640 pixels.

9.3 results.png (training curves)

Ten panels over 50 epochs: three training losses (box, class, DFL), three validation losses, and four validation metrics (precision, recall, mAP@0.5, mAP@0.5:0.95). The orange dotted line is a smoothed version.

- **All losses fall steadily.** Training box loss goes from about 1.41 to 0.70, class loss from about 2.2 to 0.48, DFL from about 1.51 to 1.01. Validation box loss goes from about 1.38 to 0.90 and validation class loss to about 0.56.
- **Metrics rise steadily.** Precision 0.54 → 0.89, recall 0.41 → 0.73, mAP@0.5 0.41 → 0.80, mAP@0.5:0.95 0.23 → 0.56.
- **Epoch 4 spike.** Validation losses jump (box loss to about 1.59, class loss to about 2.1) and the metrics dip (precision about 0.48, mAP@0.5 about 0.34) before recovering at epoch 5. This is typical of the warm-up phase, when the learning rate and bias learning rate are high and the freshly initialised class head is still adapting. It recovered by itself, so no action was needed.
- **Drop near epoch 41.** The training class and box losses step down. That is where `close_mosaic=10` switches off mosaic augmentation: images become easier and more realistic, so training loss falls.
- **No over-fitting.** The validation loss keeps decreasing and does not turn upward; the gap to the training loss is small. Test results match validation.
- **Under-training.** Every metric is still climbing at epoch 50 and nothing has plateaued, so more epochs (for example 100–150) would probably raise accuracy. This is an expectation, not something you have run.

9.4 BoxF1_curve.png (F1 versus confidence)

F1 is the balance of precision and recall at each confidence threshold. The thick blue curve (all classes) peaks at **F1 = 0.80 at confidence 0.361**. That is the best single threshold for this model. The curve is flat between about 0.2 and 0.6, so the system is not fragile to the exact threshold in that range. Approximate per-class peaks: Machinery 0.90, Mask 0.88, Person 0.86, NO-Safety Vest 0.84, Hardhat 0.82, Safety Vest and NO-Hardhat 0.79, Vehicle 0.77, NO-Mask 0.72, Safety Cone 0.64. Every curve collapses above about 0.8 because few predictions have such high confidence. Use a threshold near 0.35, and tune it on validation data, not test data.

9.5 BoxP_curve.png (precision versus confidence)

Precision rises as the threshold rises. For all classes it is about 0.9 near a confidence of 0.4 and about 0.97 near 0.6, reaching 1.00 at the maximum threshold (the legend reads 1.00 at 1.000), which is trivial because almost nothing survives that threshold. Practical meaning: if you need very few false alarms, raise the threshold, and accept more misses.

9.6 BoxR_curve.png (recall versus confidence)

Recall falls as the threshold rises. The all-class curve starts at **0.85 at confidence 0** (the legend: 0.85 at 0.000). That is the recall **ceiling**: even accepting every detection, about 15 percent of objects are never found. At the F1-optimal confidence of about 0.36 recall is roughly 0.74. Safety Cone starts at only about 0.71 and NO-Mask at about 0.78, which confirms these two are hard to find at any threshold.

9.7 BoxPR_curve.png (precision–recall)

Precision stays near 1.0 until recall reaches about 0.4–0.5, then drops. The area under each curve is that class's AP@0.5, shown in the legend: Machinery 0.926, Mask 0.892, Person 0.888, NO-Safety Vest 0.851, Hardhat 0.808, Safety Vest 0.795, NO-Hardhat 0.782, Vehicle 0.755, NO-Mask 0.691, Safety Cone 0.596, and **all classes 0.798 mAP@0.5**. A curve that stays high and far to the right (Machinery) is a strong class. A curve that bends early and left (Safety Cone) is weak. The Safety Cone curve ends at a recall near 0.71, matching its ceiling in the recall curve.

9.8 confusion_matrix.png (raw counts)

Rows are what the model predicted and columns are the true class. The diagonal holds correct detections: Person 1,629; Machinery 1,033; NO-Safety Vest 599; NO-Mask 469; Hardhat 467; Safety Vest 455; Safety Cone 552; NO-Hardhat 288; Mask 273; Vehicle 237.

- The **bottom "background" row** counts misses: for example Safety Cone 207, NO-Mask 120, Safety Vest 96, Person 96, Hardhat 87.
- The **right "background" column** counts detections where there is no real object. The numbers are very large (Person 16,623, Safety Cone 7,208). That tells you this matrix was built with a very low confidence threshold, so it includes thousands of faint, low-confidence boxes that a deployed system would discard at a threshold of about 0.36. Do not present those counts as the false-alarm rate of the system. (This is my reading of the figure; the count scale itself shows it.)

9.9 confusion_matrix_normalized.png

Each column is divided by the number of true objects in that class, so the diagonal is the recall at that setting: Machinery 0.91, Person 0.81, Mask 0.81, Vehicle 0.75, NO-Safety Vest 0.71, NO-Mask 0.70, Safety Cone 0.68, Hardhat 0.66, Safety Vest 0.66, NO-Hardhat 0.59.

- **Biggest class confusion:** 18 percent of true NO-Hardhat objects were predicted as Hardhat (86 of 486). Both look similar at a distance – a head with or without a helmet.
- **Vest confusion:** about 8 percent of Safety Vest objects were predicted as NO-Safety Vest and about 8 percent in the other direction; vests are also sometimes predicted as Person (about 8 percent).
- **Mask versus NO-Mask:** about 6–7 percent confused in each direction.
- **Vehicle versus Machinery:** about 7 percent of vehicles were predicted as Machinery.
- **Misses (predicted background):** Safety Cone 25 percent, NO-Mask 18 percent, Safety Vest 14 percent, NO-Hardhat 13 percent, Hardhat 12 percent, Vehicle 11 percent.
- The right-hand "background" column in this version shows the share of all false positives that falls on each class: Person 27 percent, Safety Cone 12 percent, NO-Safety Vest 11 percent.

Take-away sentence for the viva: "The detector is precise but misses small and ambiguous objects; the main class-to-class errors are NO-Hardhat taken for Hardhat and vest versus no-vest, which matter most for a PPE system and are the first thing I would fix with more data and a larger input size."

9.10 Speed

From the log, on the T4 with batch 16: validation 0.6 ms pre-processing, 5.9 ms inference, 2.0 ms post-processing per image; test 2.1, 11.4 and 2.7 ms. That is roughly 60 to 120 images per second in batched evaluation. This is **not** a live-video FPS: it excludes video decoding, tracking, rule logic and the dashboard, and the two runs differ by a factor of two (warm-up and disk reads from Drive are likely causes). Measure end-to-end FPS on your demo machine and quote that number.

10. The monitoring system built around the detector

This section is taken from your project report, because the application code was not uploaded. **(Confirm each number and name in your code.)**

Pipeline. Frame from webcam, video or uploaded image → YOLO11s detection → ByteTrack tracking → zone test and dwell timer → PPE association → rule engine → event record with snapshot → Flask dashboard.

ByteTrack in one minute. ByteTrack associates detections between consecutive frames. It uses a Kalman filter to predict where each tracked object will be and matches predictions to detections by IoU. Its key idea is to also use **low-confidence** detections in a second matching pass, which recovers objects that are briefly partly hidden instead of discarding them. It does not use appearance features, so after a long occlusion the same person can receive a new ID. That is why your report calls IDs temporary and camera-local, and why you say you do not do re-identification.

Zone logic. The user draws a polygon on the camera view. For each tracked person a representative point (for example the bottom-centre of the box, which approximates where the feet are) is tested with a point-in-polygon check. While the point is inside, a dwell timer accumulates; when it reaches the 1.5-second threshold an event is created. The threshold exists to ignore someone who merely brushes the edge of a zone or a one-frame misdetection, which is the same logic Zaidi et al. [6] use with a time window.

PPE association. PPE detections (Hardhat, Safety Vest, Mask and their NO- versions) are matched to a person box by spatial containment or overlap, then each required item becomes one of: detected, explicit NO- class detected, or uncertain (not visible). Uncertain is deliberately separate from missing, because a detector failing to see a helmet is not proof there is none.

Rules and events. The rule engine evaluates zone dwell, missing required PPE, machinery presence and the closing-time rule. Each event stores timestamp, zone, track ID, PPE status and an evidence image. A cooldown or state flag prevents the same continuing violation from creating a flood of duplicate alerts.

After-hours rule. If local time is at or after the configured closing time and a person is detected, an after-hours event is created and shown on the dashboard.

Check this in your code. In your training log the Ultralytics default tracker setting reads `tracktrack.yaml`, not ByteTrack. If your app calls `model.track(...)`, make sure it explicitly passes `tracker="bytetrack.yaml"` so that the tracker you describe in the viva is the tracker you actually run.

Design principles to quote. Track IDs are references, not identities. Uncertain is not the same as missing. Always keep evidence images for human review. Calibrate zones to each camera view. Prevent repeated alerts for one continuous condition.

PART C – EXPLANATION OF EVERY FILE

Files you uploaded

File	What it is	What to say about it
DL_17_10.ipynb	The Colab training and evaluation notebook (11 cells)	The single source of truth for how the model was trained: dataset download, training call, test evaluation, backup. Walk through it using the table in section 6.
SafeSight_Project_Report.docx	The written project report	Abstract, introduction, related work with base paper and gap, method and algorithm, experiments, results, conclusion and references [1]–[8]. Needs a small update before the viva (section 12).

File	What it is	What to say about it
labels.jpg	Auto-generated dataset statistics for the training split	Class counts, box overlay, centre-position heat map, width–height heat map. Shows imbalance and small objects. See section 4.1.
results.png	Training and validation curves per epoch	Ten panels: losses and metrics. Shows steady convergence, the epoch-4 dip, the mosaic-off drop near epoch 41, and no over-fitting. See 9.3.
BoxF1_curve.png	F1 versus confidence, per class	Best all-class F1 0.80 at confidence 0.361. See 9.4.
BoxP_curve.png	Precision versus confidence	Precision about 0.9 at 0.4 and about 0.97 at 0.6. See 9.5.
BoxR_curve.png	Recall versus confidence	Recall ceiling 0.85 at confidence 0. See 9.6.
BoxPR_curve.png	Precision–recall curves with AP per class	Class AP and mAP@0.5 = 0.798. See 9.7.
confusion_matrix.png	Raw-count confusion matrix	Shows exact counts of correct detections, class mix-ups and misses. See 9.8.
confusion_matrix_normalized.png	Same matrix divided by true-class totals	Per-class recall on the diagonal; clearer for comparing classes. See 9.9.

Standard files created by Ultralytics in the run folder

The notebook and report refer to these. If they are in your folder, this is what they are.

- **weights/best.pt** – the checkpoint with the highest fitness (0.1 × mAP@0.5 + 0.9 × mAP@0.5:0.95) on validation. 19.2 MB. This is the model the app should load and the one evaluated on the test set.
- **weights/last.pt** – the checkpoint after the final epoch; used to resume training.
- **results.csv** – one row per epoch with losses and metrics; the source of results.png and of the epoch-50 numbers in your report.
- **args.yaml** – every training argument that was used (the long argument line in the log); proof of your hyperparameters.
- *train_batch.jpg** – sample augmented training batches (you can see mosaic here).
- *val_batch_labels.jpg and val_batch*_pred.jpg** – ground truth versus predictions on validation images; the source of the prediction mosaic in your report.
- **labels_correlogram.jpg** – pairwise distributions of box position and size.

Application files (not uploaded – fill in the real names before the viva)

Your report describes these components. Replace the right-hand column with your actual file names and what each does.

Component	Responsibility	Your file name
Web server	Flask routes for upload, live camera, zone setup, events and dashboard data	(fill in)
Detector and tracker wrapper	Loads best.pt, runs detection and ByteTrack, returns boxes, classes and track IDs	(fill in)
Zone and rule engine	Polygon zones, point-in-polygon test, dwell timer, PPE association, cooldown, after-hours rule	(fill in)
Event store	Saves event time, zone, track ID, PPE status and snapshot path	(fill in)
Dashboard templates and static files	Live view, counts, event table with snapshots, settings	(fill in)
Evidence folder	Saved snapshot images for events	(fill in)

Component	Responsibility	Your file name
Configuration	Thresholds, zones, required PPE, closing time	(fill in)

11. What was done from scratch and what was reused

Reused (say so openly)	Your own work (say only what you personally did)
YOLO11 architecture and COCO-pretrained yolo11s.pt weights from Ultralytics	Choosing the model, the ten-class setup and the full set of training hyperparameters (section 7)
Ultralytics training loop, augmentation, loss functions, validation and plotting	Running the training on a Colab T4 and saving and backing up the results (notebook)
The Roboflow public dataset and its labels (CC BY 4.0)	Dataset verification (split counts, class names) and analysis of imbalance and small objects
The ByteTrack algorithm as implemented in Ultralytics	Evaluation on validation and test sets and interpretation of all curves and matrices
Flask, Colab, PyTorch and other libraries	**(Confirm in your code)** the zone and dwell logic, PPE-to-person association, rule engine, event records, snapshots, dashboard and after-hours rule
Ideas from the literature (Cheng et al. [3] for person-linked PPE incident logging; Zaidi et al. [6] for temporal confirmation)	Literature survey, identification of the research gap, and the project report

A safe sentence: “I did not design a new neural network or collect a new dataset. I fine-tuned a pretrained YOLO11s on a public dataset and built and evaluated a monitoring workflow around it.”

12. Limitations, improvements and a checklist to fix before the viva

Limitations (state them yourself)

1. Detector metrics are not event metrics; event-level accuracy is not measured.
2. Single training run, one seed, no hyperparameter search and no model comparison.
3. Under-trained: metrics still rising at epoch 50.
4. Weak classes (Safety Cone, NO-Mask) and a Hardhat drop on the test set.
5. Small objects at 640 pixels; the recall ceiling is 0.85.
6. Track IDs are camera-local and can change after occlusion; no re-identification.
7. PPE status depends on a spatial heuristic that can fail in crowds and with occlusion.
8. Dataset has no empty-scene images and an unusual position pattern (section 4.1); generalisation to your own camera views is untested.
9. Evidence snapshots of workers are personal data; a retention and consent policy is needed for real use.

Improvements in order of expected value

1. Train longer (100–150 epochs) and check that validation keeps improving.
2. Raise the input size (800 or 960) and/or try YOLO11m, then compare small-class AP and FPS.
3. Add data for Safety Cone, NO-Mask and Hardhat/NO-Hardhat, including hard negatives and empty scenes, and balance classes.
4. Choose per-class confidence thresholds on validation data, leaning toward recall for safety-critical classes.
5. Temporal smoothing of PPE status over several frames (Zaidi et al. [6]).
6. Crop each person and run a dedicated PPE classifier, as in Cheng et al. [3], to reduce association errors.
7. Annotate a short set of videos and measure the event-level metrics listed in section 8.
8. Cross-camera re-identification, if multi-camera operation is wanted.

Fix before the viva (the report is slightly out of date)

1. **Test metrics now exist.** Your report says no test-set output was supplied, but the notebook shows it: precision 0.889, recall 0.737, mAP@0.5 0.805, mAP@0.5:0.95 0.567 on 260 images. Add the table from section 9.2 and change the wording in the abstract,

results and conclusion.

2. **Hardware is identifiable.** The report says the hardware cannot be identified; the log shows a Tesla T4 (14,913 MiB) and 0.445 hours of training. Update section 4.2.
3. **Speed numbers exist** for batched evaluation (section 9.10). Keep the caveat and add your measured live-video FPS.
4. **Tracker.** Confirm that the app explicitly uses ByteTrack (section 10).
5. **Notebook tidy-up.** Remove the duplicate installs (cells 0 and 1), the duplicate data.yaml check (cell 5), and the unused `device` variable; note that `pretrained=True` is redundant. None of these changes the results.
6. **Credit the dataset** (CC BY 4.0) on a slide.
7. **Rehearse the demo twice** and keep a recorded fallback.

PART D – VIVA QUESTIONS WITH ANSWERS AND CROSS-QUESTIONS

How to answer: give the direct answer in one or two sentences first, then the reason, then stop. If you do not know, say what you do know and what you would check; never invent a number. Cross-questions are the follow-ups a faculty member is likely to ask when your first answer is correct but they want to test depth.

D1. Basics and motivation

Q1. Explain your project in one minute. A. SafeSight is a camera-based construction-site safety monitor. I fine-tuned a YOLO11s object detector on ten safety classes (people, PPE and its absence, machinery, vehicles, cones) and built a monitoring layer around it: tracking, user-drawn zones, a dwell-time rule, PPE assessment per person, evidence snapshots, an after-hours rule and a live Flask dashboard. The detector reaches mAP@0.5 of about 0.80 on validation and test data. The main contribution is the integrated workflow and a plan to evaluate it at event level.

Q2. Why is this problem important? A. Construction is high-risk and PPE reduces injuries, but one supervisor cannot watch every area all shift. Automated detection gives consistent, continuous checking and keeps evidence for review.

Cross-Q. Will this replace safety officers? A. No. It flags conditions for human review. The system is designed to keep evidence images so a person decides, and my report states it is not a validated safety product.

Q3. Is this classification, detection or segmentation? Why detection? A. Detection. One image contains many objects of different classes and I need each one's location and class. Classification gives one label per image; segmentation gives per-pixel masks, which is more expensive than needed for boxes.

Q4. Why ten classes and not just hardhat and vest? A. The positive and negative PPE classes let the model detect violations directly, Person anchors each worker, and Machinery, Vehicle and Safety Cone give hazard context for rules like machinery presence and zones.

Cross-Q. What does the Mask class mean here? A. In this dataset it refers to face masks (confirm by viewing sample images before the viva). It is the same positive/negative design as hardhat and vest.

Q5. What are your objectives? A. Train and use a YOLO11s detector for ten classes; connect live or uploaded imagery to camera-local tracking and zone rules; record time, zone, track ID, PPE status and evidence for each event; show detections and events on a Flask dashboard; and report detector results while identifying the event-level evaluation still needed.

D2. Literature, base paper and novelty

Q6. Which is your base paper and why? A. Cheng et al. 2022 in Automation in Construction, "Vision-based monitoring of site safety compliance based on worker re-identification and PPE classification." It combines re-identification and PPE classification and logs incidents of workers without PPE, which is the closest system-level idea to my event record that links a person's PPE state to a log entry.

Cross-Q. What did they do that you did not? A. Cross-camera worker re-identification, so a worker can be recognised across different views. I only use ByteTrack IDs that are valid within one camera stream.

Cross-Q. What did you do that is different? A. Based on their abstract, I add user-drawn polygon zones with a dwell rule, an after-hours rule, evidence snapshots and a live dashboard, and I detect violations with explicit NO-classes from a single-stage detector. I have read the abstract closely and I should not claim more than that about their internals.

Cross-Q. Did you reproduce their results? A. No. Their 4 percent and 13 percent accuracy improvements are for their own dataset and tasks. I do not compare my mAP to those numbers.

Q7. What is the research gap? A. PPE detection, person-linked PPE logging and temporal false-alarm reduction each exist in the literature. The gap I address is narrow: a single-camera prototype combining user-drawn zones, dwell time, per-person PPE evidence, snapshots, a closing-time rule and a dashboard, and the event-level evaluation of that combination, which is missing.

Cross-Q. Isn't this just a standard YOLO project with a dashboard? A. The detector part is standard fine-tuning, and I say so. The contribution is the integration, the explicit handling of uncertain versus missing PPE, and the evaluation plan. I am not claiming a new architecture or state-of-the-art accuracy.

Cross-Q. Then what exactly is novel? A. Nothing broad. The defensible claim is the specific combination and its measurement. A broader novelty claim would need a systematic review and a comparison experiment, which I have not done.

Q8. Name the other papers and what you took from each. A. Delhi et al. 2020: YOLOv3 PPE compliance with alarms and reports (F1 0.96 on four compliance categories), the operational precedent. Wang et al. 2021: eight YOLO detectors on the CHV dataset, YOLOv5x best mAP 86.55 percent, YOLOv5s fastest at 52 FPS, supporting the accuracy-speed trade-off. Zaidi et al. 2024: temporal analysis across frames to cut false alarms, supporting my dwell rule. Karlsson et al. 2023: distance effects. Al-Khiami and ElHadad 2024: custom Ultralytics YOLOv8 precedent. Tong et al. 2025 and Li et al. 2025: efficient and small-object PPE detection, which inform my future work.

Cross-Q. Wang et al. got 86.55 percent mAP and you got 80 percent. Is yours worse? A. The numbers are not comparable. They used six classes on their own 1,330-image dataset; I use ten classes, including hard small classes such as Safety Cone, on a different dataset. A fair comparison needs the same dataset and protocol.

Cross-Q. Delhi et al. got F1 of 0.96; why is yours 0.80? A. Their task was four-way compliance classification for two PPE items; mine is ten-class object detection, which also requires localisation. Different tasks, different difficulty.

Q9. Why do you use a dwell time? A. A single-frame detection of a person near a zone can be noise or a brief edge crossing. Requiring the person to stay in the zone for 1.5 seconds filters transient detections. Zaidi et al. use the same idea with a time window to cut false alarms.

D3. Dataset

Q10. Where did you get the data? Did you collect or label it yourself? A. It is a public Roboflow project, version 1, licence CC BY 4.0, downloaded in YOLO format. I did not collect or label it myself.

Cross-Q. Is 2,597 images enough? A. It is small, but transfer learning from COCO weights makes it workable and test results match validation. It is not enough to claim the model generalises to a different site or camera; that would need data from the target site.

Q11. How is the data split and why? A. 1,819 train, 518 validation and 260 test, about 70/20/10. Train updates the weights, validation tracks progress and selects best.pt, test is used once at the end.

Cross-Q. Why do you need both validation and test? A. Because choosing best.pt on validation makes validation slightly optimistic. The untouched test set gives an unbiased estimate. Mine agree (0.798 versus 0.805), which is reassuring.

Cross-Q. Could there be data leakage between splits? A. Roboflow exports can include augmented copies, and leakage would inflate results if copies of the same image sit in different splits. I should check the dataset version page to confirm augmentation was applied to the training split only. I have not verified this, so I treat the test score with some caution.

Q12. What is the label format? A. One text file per image; each line is class index, x-centre, y-centre, width, height, all normalised by image size. data.yaml lists splits, nc = 10 and class names.

Q13. Is the dataset balanced? What did you do about it? A. No. Person has 6,715 boxes and Vehicle 1,065, about 6.3 to 1. I did not apply class weighting or oversampling; mosaic augmentation helps somewhat. Imbalance is not the main driver of the weak classes though: Safety Cone has 1,902 boxes but the lowest AP, so size and appearance matter more.

Q14. What does labels.jpg show? A. Class counts, an overlay of all boxes, a heat map of box centres and a heat map of box width versus height. The last shows that most objects are small, under about 10 percent of image width and height.

Cross-Q. Why are the centres clustered in four blobs? A. I am not certain from the plot alone. It may reflect how the source images were composed or tiled. I would inspect random training images to confirm. It does not change the training procedure.

Q15. What if a head is visible but neither Hardhat nor NO-Hardhat is detected? A. My design treats that as uncertain, not as a violation, because a detector failing to see an item is not proof it is absent.

D4. Model choice and architecture

Q16. Why YOLO11s? A. It is a single-stage detector that is fast enough for live video, small (9.4 M parameters), comes with COCO-pretrained weights, and Ultralytics provides training, validation and ByteTrack in one library. Wang et al. show smaller YOLO models trade a little accuracy for much more speed.

Cross-Q. Did you compare with YOLOv8 or YOLO11m? A. No. The choice rests on the literature and design constraints. A good next experiment is YOLO11s versus YOLO11m and YOLOv8s on the same split, comparing mAP, small-class AP, FPS and memory.

Cross-Q. Why not Faster R-CNN? A. Two-stage detectors are strong but heavier and slower for live video, and my system needs real-time operation on modest hardware.

Q17. Explain the YOLO11 architecture briefly. A. Backbone with strided convolutions and C3k2 blocks plus attention (C2PSA) extracts features; a neck fuses three scales; a decoupled, anchor-free head predicts boxes and classes separately on 80×80, 40×40 and 20×20 grids for a 640 input, giving 8,400 candidates that go through confidence filtering and NMS.

Cross-Q. What does anchor-free mean? A. The head predicts box edges directly relative to each grid cell, so I do not need pre-defined anchor box shapes tuned to the dataset.

Cross-Q. What are the three losses? A. A CIoU-type box loss for overlap and shape, a distribution focal loss for fine box edges, and binary cross-entropy for classes, weighted 7.5, 1.5 and 0.5 (box, DFL, class).

Q18. What is transfer learning and why did it work? A. I start from weights learned on COCO and fine-tune. Early layers already encode edges, textures and shapes and COCO contains people, so the model reaches 0.41 mAP@0.5 after the first epoch with only 1,819 images.

Cross-Q. COCO has no hardhat class, so why does it help? A. The reusable part is general visual features, not the class labels. The output layer is re-sized to ten classes and fine-tuned.

Q19. Did you build the model from scratch? A. No. I did not design a neural network or train from random weights. I fine-tuned a pretrained model and built the system around it.

Q20. What is NMS and what is IoU? A. IoU is overlap area divided by union area of two boxes. NMS keeps the highest-confidence box among overlapping boxes of the same class (IoU limit 0.7 in my log) to remove duplicates.

D5. Training and hyperparameters

Q21. Walk me through how you trained the model. A. In Colab I installed Ultralytics and Roboflow, downloaded the dataset in YOLO format with the API key from Colab Secrets, checked the split and class names, loaded yolo11s.pt and called `model.train` for 50 epochs at 640 pixels, batch 16, SGD with learning rate 0.01, on a Tesla T4. It took about 27 minutes. Each epoch it trained on augmented images, validated on 518 images, saved last.pt and saved best.pt whenever fitness improved. Afterwards I loaded best.pt, evaluated it on the 260 test images and backed up the run folder from Drive to Drive.

Q22. Where did you train and why? A. Google Colab with a Tesla T4 (about 15 GB). It is free or cheap, needs no local GPU and Drive keeps the weights if the session disconnects.

Cross-Q. Limitations of Colab? A. Sessions can time out and hardware is not guaranteed, so I saved outputs to Drive and made a backup. The speed numbers I measured are for that T4, not for my demo machine.

Q23. Why 50 epochs? A. It was a time-budgeted choice that reached about 0.80 mAP@0.5 in 27 minutes. My curves show the metrics still improving at epoch 50, so the model is under-trained; I would train 100 to 150 epochs and watch validation for the plateau.

Cross-Q. Why did early stopping not trigger? A. Patience was 100 epochs, larger than the whole run, and the metrics were still improving anyway.

Q24. Why SGD and not Adam? A. SGD with momentum is the classic recipe for YOLO and tends to generalise well; Adam-type optimisers often converge faster in short runs. I chose SGD explicitly and did not compare optimisers, so I cannot say which would have been better here.

Cross-Q. What would happen with a higher learning rate? A. Faster early progress but a risk of instability, like the dip at epoch 4. Too low would make convergence slower, which matters because my run is short.

Q25. Explain lr0, lrf and momentum. A. lr0 = 0.01 is the starting learning rate. lrf = 0.01 means the final learning rate is lr0 times lrf, which is 0.0001; the schedule decays linearly. Momentum 0.937 keeps most of the previous update direction, smoothing and accelerating descent.

Q26. Why weight decay 0.001? A. It is an L2 penalty on weights that discourages over-fitting. 0.001 is twice the usual 0.0005, a deliberately stronger regulariser for a small dataset. I did not run an ablation to prove it was better.

Q27. Why batch size 16 and image size 640? A. Batch 16 fits a 15 GB T4 comfortably with mixed precision, and gradients are accumulated to a nominal batch of 64 before each weight update. 640 is the standard YOLO input size; a larger size would help small objects like cones but would be slower and use more memory.

Q28. What augmentations were used? A. The Ultralytics defaults: mosaic (four images combined), HSV colour jitter, 10 percent translation, 50 percent scale change, horizontal flips, random erasing, RandAugment, and small chances of blur, grayscale and contrast enhancement. No mixup, rotation or vertical flip.

Cross-Q. What is mosaic and why switch it off for the last 10 epochs? A. Mosaic tiles four training images into one, giving objects at varied scales and contexts. It is unrealistic compared with real frames, so switching it off at the end lets the model fine-tune on natural images. You can see this as a step in the training losses near epoch 41.

Cross-Q. Your log says augment=False. Did you train without augmentation? A. No. That flag is test-time augmentation for inference. Training augmentation is controlled by mosaic, hsv, scale and so on, and it was on.

Q29. What happened at epoch 4 in results.png? A. The validation losses spiked and mAP@0.5 dipped to about 0.34 before recovering at epoch 5. This is typical of the warm-up phase where learning rates are high and the new class head is still adjusting. It resolved without intervention.

Q30. Is your model over-fitted? A. Not on the evidence I have. Validation loss keeps falling, the gap to training loss is small and test mAP (0.805) is slightly higher than validation (0.798). It is more likely under-trained.

Cross-Q. Then why is mAP@0.5:0.95 only 0.56? A. That metric demands tight boxes at IoU up to 0.95. Small objects at 640 pixels are hard to box precisely, so it is naturally much lower than mAP@0.5.

Q31. What is AMP and does it affect accuracy? A. Automatic mixed precision runs parts of the network in 16-bit to save memory and time. Ultralytics checked it at the start (the check passed), and it normally has negligible effect on accuracy.

Q32. How is best.pt chosen and is it the last epoch? A. By a fitness score of 0.1 times mAP@0.5 plus 0.9 times mAP@0.5:0.95 on validation. Because metrics were still rising, the best checkpoint is at or very near the final epoch.

Cross-Q. Would your results change if you retrained? A. Slightly. I fixed the seed to 0 and enabled deterministic mode, but GPU operations and library versions can still cause small differences. I ran once, so I cannot give a confidence interval.

D6. Evaluation and results

Q33. Define precision, recall, F1, AP and mAP. A. Precision is TP over TP plus FP; recall is TP over TP plus FN; F1 is their harmonic mean. AP is the area under a class's precision–recall curve and mAP is the mean AP over classes. mAP@0.5 uses IoU 0.5; mAP@0.5:0.95 averages over IoU 0.50 to 0.95 in steps of 0.05.

Cross-Q. Why not use accuracy? A. Detection has no natural count of true negatives (almost every possible box is background), so accuracy is not meaningful. Precision, recall and AP handle localisation and class together.

Cross-Q. Does mAP 0.80 mean the system is 80 percent accurate? A. No. It is the average area under per-class precision–recall curves at IoU 0.5 for single images. It says nothing directly about zone alerts, tracking or PPE-to-person matching.

Q34. State your results. A. Validation: P 0.891, R 0.728, mAP@0.5 0.798, mAP@0.5:0.95 0.561. Test: P 0.889, R 0.737, mAP@0.5 0.805, mAP@0.5:0.95 0.567. Best F1 is 0.80 at a confidence of 0.361.

Q35. Why is recall lower than precision? A. At the operating point the model is conservative: it rarely reports things that are not there but misses small, occluded or ambiguous objects. The confusion matrix and the recall ceiling of 0.85 show many misses on small classes.

Cross-Q. How would you improve recall? A. Lower the confidence threshold for safety-critical classes, increase the input size, train longer, add data for weak classes and consider a larger model. Each has a cost: more false alarms, lower speed or more memory.

Q36. Which classes are strongest and weakest, and why? A. Strongest: Machinery (AP 0.926), Mask and Person. Weakest: Safety Cone (0.596) and NO-Mask (0.691). Cones are small, often far away and appear in clusters; NO-Mask depends on a small, subtle face region and subjective labelling. Class count is not the reason: cones have more examples than masks.

Cross-Q. What would you do about Safety Cone? A. Add more varied cone images (distance, lighting, clutter), use a higher input size or tiled inference, and treat cone detections cautiously in rules.

Q37. Why did Hardhat accuracy drop on the test set? A. AP@0.5 went from 0.808 to 0.669 and recall to 0.59. It may be due to the small test sample, different image conditions in the test split and the Hardhat versus NO-Hardhat confusion (18 percent of NO-Hardhat objects are predicted as Hardhat). These are hypotheses; I did not run an experiment to isolate the cause.

Q38. Explain the confusion matrix. A. Rows are predictions and columns are true classes. The diagonal is correct detections. The bottom background row is missed objects and the right background column is detections with nothing real behind them. The normalised version divides by true-class totals, so the diagonal is recall: Machinery 0.91, Person 0.81, down to NO-Hardhat 0.59.

Cross-Q. Why are the background counts so large (Person 16,623)? A. The matrix was evidently computed with a very low confidence threshold, so it counts thousands of faint predictions that would be removed at the operating threshold of about 0.36. It should not be read as the deployed false-alarm rate.

Cross-Q. What are the most important confusions? A. NO-Hardhat predicted as Hardhat (18 percent), Safety Vest versus NO-Safety Vest (about 8 percent each way), and Mask versus NO-Mask (6–7 percent). They matter because they directly flip a compliance decision.

Q39. How do you choose the confidence threshold? A. From the F1–confidence curve: the all-class F1 peaks at 0.80 at 0.361, and the curve is flat from about 0.2 to 0.6. I would tune it on validation data, and per class if needed, never on the test set. For a safety system I would lean toward a lower threshold for critical classes to favour recall.

Q40. What do the precision and recall curves tell you? A. Precision rises with threshold (about 0.9 at 0.4, 0.97 at 0.6) while recall falls. Recall at threshold 0 is 0.85, so 15 percent of objects are never detected regardless of threshold.

Q41. What does the PR curve show? A. Precision against recall for each class; the area under each curve is its AP. Machinery's curve stays high and far right (0.926), Safety Cone's bends early (0.596). The bold curve is the all-class mean, mAP@0.5 = 0.798.

Q42. Is 0.80 mAP good? A. For ten classes including small, ambiguous ones, with 1.8 thousand training images and 27 minutes of training, it is a reasonable baseline and still improving. I do not claim it beats other papers because the datasets and tasks differ. Whether it is good enough depends on event-level requirements, which I have not measured.

Q43. What is your speed? Is it real-time? A. In batched evaluation on the T4, inference is about 6 to 11 ms per image plus 2 to 3 ms for pre- and post-processing, so the detector itself is fast enough. The full system adds decoding, tracking, rules and the dashboard, so I report the end-to-end FPS measured on the demo machine and do not claim real-time beyond it.

Q44. How confident are you in these numbers statistically? A. Limited. It is one run with one seed, and the test set has 260 images, so per-class test numbers (for example Mask with 164 objects, Vehicle with 153) have wide uncertainty. Repeated runs or cross-validation would be needed for confidence intervals.

D7. The monitoring system

Q45. How do zones work? A. The user draws a polygon on the camera view. For each tracked person I take a representative point, for example the bottom-centre of the box, and test whether it is inside the polygon. While inside, a dwell timer accumulates; at the threshold (1.5 seconds in my report) an event is created. **(Confirm in code.)**

Cross-Q. Why the bottom-centre and not the box centre? A. The feet determine where a person stands on the ground plane. A box centre of a tall box near a zone edge can fall inside the polygon while the person is outside it.

Cross-Q. How did you choose 1.5 seconds? A. It is a configurable default to ignore momentary detections and brief edge crossings, motivated by the temporal-analysis idea in the literature. I did not optimise it on labelled video, and I list that as part of the missing event-level evaluation.

Q46. What is ByteTrack and why use it? A. A multi-object tracker that matches detections between frames using motion prediction (Kalman filter) and IoU. It also uses low-confidence detections in a second pass, so briefly hidden people are not lost. It is built into Ultralytics, fast, and needs no extra appearance network.

Cross-Q. What are its limitations? A. It has no appearance model, so after long occlusion or crossing paths the same person may get a new ID. That could restart a dwell timer or double-count a person, which is why IDs are temporary and camera-local, and why I say I do not do re-identification.

Q47. How do you decide whether a person is wearing PPE? A. PPE detections are matched to each person box by spatial containment or overlap. Each required item becomes detected, explicit NO-class detected, or uncertain. A missing detection is uncertain, not a violation.

Cross-Q. What fails in a crowd? A. Overlapping people can cause a hardhat or vest box to be matched to the wrong person. That is a known weakness; a person-crop classifier or pose-based assignment, as in other research, would reduce it.

Q48. How do you avoid repeated alerts and false alarms? A. The dwell threshold filters transient detections, and event state or a cooldown stops the same continuing condition from creating many events. Each event has a snapshot so a human can dismiss false ones. I have not yet measured false alerts per camera-hour.

Q49. What is the after-hours rule? A. If the local time is at or after the configured closing time and a person is detected, an after-hours event is created and shown on the dashboard. **(Confirm in code how time zone and sustained detection are handled.)**

Q50. What does an event contain? A. Timestamp, zone, track ID, observed PPE status and an evidence image, shown on the dashboard.

D8. Honesty, ethics, deployment and future work

Q51. Is this ready for a real construction site? A. No. It is a prototype. The detector has useful accuracy, but event-level reliability, false-alarm rate, latency and multi-site generalisation are not yet measured.

Q52. What if the camera angle, lighting or weather is different? A. Accuracy may drop. The model was trained on the dataset's images, and I have not tested night, rain, glare or other camera views. Zones must be redrawn per camera, and fine-tuning on site images would be the next step.

Q53. What about privacy? A. Evidence snapshots show workers, which is personal data. A real deployment needs consent, access control, a retention limit and a clear purpose. The dataset is CC BY 4.0, so I credit its authors.

Q54. What are the main limitations? A. Detector metrics are not event metrics; one run and no hyperparameter search; under-trained; weak small-object classes and the Hardhat test drop; camera-local temporary IDs; heuristic PPE association; dataset has no empty scenes and limited site variety.

Q55. What would you do with more time? A. Train 100 to 150 epochs at a larger image size and compare YOLO11m; add data and balance for cone, NO-Mask and hardhat classes; choose per-class thresholds; add temporal smoothing of PPE status; annotate videos to measure event precision, recall, false alerts per hour, ID switches, association accuracy and latency; and consider re-identification for multi-camera use.

Q56. What did you learn? A. That a good detector metric is only the start: the rules and tracking around it decide whether alerts are trustworthy, and that being explicit about uncertainty and limits matters as much as the headline number.

Q57. A detector with 73 percent recall misses a quarter of objects. Would you trust it for safety? A. Not as the only safeguard. It should assist human supervisors, with thresholds tuned toward recall for critical classes and with evidence review. The weakness is why event-level evaluation and temporal confirmation are my first future-work items.

Q58. Who made YOLO11 and which of this work is yours? A. YOLO11 is from Ultralytics. My work is the training configuration and run, the evaluation and analysis, the literature survey and gap, and, per my report, the monitoring layer: zones, dwell logic, PPE association, events, snapshots, dashboard and after-hours rule. **(Say only what you personally wrote.)**

Rapid-fire answers (one line each)

Question	Answer
Base paper?	Cheng et al. 2022, Automation in Construction 139:104312
Dataset and licence?	Roboflow construction-site-safety v1, CC BY 4.0
Split?	1,819 / 518 / 260 images
Number of classes?	10
Model and size?	YOLO11s, about 9.4 M parameters, 21.4 GFLOPs fused
Pretrained on?	COCO (yolo11s.pt)
Optimiser and LR?	SGD, 0.01 decaying linearly to 0.0001
Epochs, image size, batch?	50, 640, 16
GPU and time?	Colab Tesla T4, 0.445 h
Validation mAP@0.5 / 0.5:0.95?	0.798 / 0.561
Test mAP@0.5 / 0.5:0.95?	0.805 / 0.567

Question	Answer
Best threshold?	0.361 (F1 0.80)
Recall ceiling?	0.85
Best / worst class?	Machinery 0.926 / Safety Cone 0.596
Biggest confusion?	NO-Hardhat predicted as Hardhat, 18 percent
Why mosaic off at end?	Fine-tune on realistic images; step at epoch 41
What does augment=False mean?	Test-time augmentation off, not training
Tracker?	ByteTrack, camera-local IDs
Dwell threshold?	1.5 s (confirm in code)
Main limitation?	No event-level evaluation yet

If you get stuck in the viva

1. Restate the question in your own words to buy time.
2. Answer with what you measured, then label anything else as a hypothesis (“I think… but I have not tested it”).
3. Point to the figure that supports the answer (F1 curve, confusion matrix, results.png).
4. If it is about code you did not write or cannot recall, say what the component is supposed to do and offer to show the file.
5. Never defend a claim the evidence does not support; naming a limit yourself is stronger than being caught on it.